Predicting Apartment Sale Prices in Queens, NY

Final Project for Math 342W Data Science at Queens College
5/26/2026

By Isa Nawaz
In collaboration with:
Padma Haran
Samiha Haque

Abstract

This paper presents a predictive model for apartment sale prices in Queens, New York, built on a dataset of condominium and co-operative unit sales between February 2016 and February 2017. The unit of observation is an individual apartment sale, and the response variable is sale price in dollars, capped at $1,000,000 and restricted to Queens zip codes. After filtering, the working dataset consists of 521 observations - a number that turns out to be the central constraint throughout the entire project.

The raw data was collected via Amazon Mechanical Turk and required substantial cleaning before any modeling could begin. Many features contained formatting inconsistencies, string-encoded dollar values, typographical errors in categorical fields, and extensive missingness - particularly in square footage (~60% missing), total taxes (~74% missing), parking charges (~75% missing), and percent tax deductible (~82% missing) before removing rows without a y. Missing values were imputed using missForest applied to the full 2,230-row dataset before filtering, in order to take advantage of information present in rows without a sale price y. Missingness indicator dummy variables were retained as features, since missingness in this dataset is structurally meaningful rather than random. The response variable was log-transformed prior to modeling, as raw sale prices showed the right-skewed distribution typical of real estate data. Predictions are transformed back to dollar scale via exponentiation for reporting in dollar terms.

Three model classes were fit and evaluated: a regression tree, OLS linear regression, and random forest with and without mtry tuning. All models were trained on a 90/10 random split of the 521 observations, with seed set to 1984 for reproducibility. The best-performing model was OLS, achieving an out-of-sample R^2 of 0.846, a log-scale *RMSE* of 0.215, and a dollar-scale *RMSE* of approximately $79,398. The best model's median absolute percent error on the test set was 12.1%, with 19.2% of predictions within 5%, 42.3% within 10%, and 65.4% within 20% of true sale price.

At first glance, it is somewhat surprising that OLS outperformed RF, since RF is usually considered the stronger predictive method. However, with only 469 training observations spread across 86 features after dummification, random forest faced a data scarcity problem. Each bootstrap sample in a 1,000-tree forest is drawn from an already small pool, and the model cannot reliably learn non-linear structure under these conditions. OLS handles this regime more efficiently, and the signal in this dataset appears to be predominantly linear - location, property type, square footage, bedroom count, and monthly carrying cost account for the bulk of predictive power. Relative to the Zillow benchmark of 2.3% median error and 80.2% within 5%, our model falls substantially short, a gap explained primarily by sample size, missingness in key features, and the inherent noise of MTurk-collected data. Despite this, the model demonstrates that the core price signal in Queens apartment data is learnable from a modest feature set, and that a well-specified OLS model remains a competitive baseline when data is limited.

1. Introduction

The goal of this project is to build a model that predicts the sale price of apartments in Queens, NY for the period between February 2016 and February 2017. The population of interest is condominiums and co-operative units sold within this window at prices at or below $1,000,000, located in the zip codes specified for the project. These zip codes cover mainland Queens and exclude the Rockaways, a geographically distinct peninsula near JFK airport.

A predictive model takes in observable characteristics of a unit - its size, location, number of bedrooms, monthly costs, etc., and outputs an estimate of what that unit would sell for. Formally, we seek a function g such that $g(x) \approx y$, where x is a vector of features describing the apartment and y is the log sale price. The model is trained on historical data $D = \{(x_1, y_1), \ldots, (x_n, y_n)\}$ and evaluated on a held-out set of test observations it has never seen, giving an honest estimate of how it would perform on future predictions.

The response variable is log sale price. As discussed in Section 2, the raw sale price distribution is right-skewed, and modeling on the log scale improves the distributional properties of residuals across all three model classes. Three modeling approaches were explored: regression tree, OLS, and random forest. OLS emerged as the best-performing model on this dataset, a result that most likely comes down to the small sample size relative to the number of features. The Zillow benchmark - 2.3% median error, 80.2% within 5% - serves as the external comparison point.

2. The Data

The dataset was collected via Amazon Mechanical Turk as a raw download from the MLSI platform, containing 2,230 rows and 55 columns before any filtering. After restricting to Queens zip codes, property types condo and co-op, sale prices at or below $1M, and rows with a recorded sale price, the working dataset consists of 521 observations. This is a small dataset by machine learning standards and is the most important constraint on model performance throughout the project.

The dataset is reasonably representative of the population of interest - Queens condo and co-op sales in the specified period - though MTurk data collection introduces quite a bit of noise and inconsistency. There are no obvious extreme outliers after the $1M cap is applied, though the distribution of sale prices is right-skewed, with a cluster of lower-priced co-ops and a tail of higher-priced condos; this led to the logging of the sale price data. Extrapolation risk exists for units with characteristics well outside the training distribution - particularly very large apartments, newly constructed buildings, and zip codes with very few observations in the dataset.

2.1 Featurization

After cleaning, the following features were used in modeling. Continuous features: log square footage (*ln_sq_footage*), building age in years (*bldg_age*, derived as 2017 minus approximate year built), number of bedrooms (*num_bedrooms*), total bathrooms combining full and half baths (*bath_total = num_full_bathrooms + 0.5 × num_half_bathrooms),* total rooms (*num_total_rooms*), number of floors in building (*num_floors_in_building*), monthly carrying cost in dollars (*monthly_cost*, merging maintenance cost for co-ops and common charges for

condos), total taxes (*total_taxes*), walk score (*walk_score*), parking charges (*parking_charges*), and percent tax deductible (*pct_tax_deductibl*). Categorical features: property type (*coop_condo*), zip code as a 42-level factor (*zip_f*), kitchen type (*kitchen_type*: eat in, efficiency, combo, none), dining room type (*dining_room_type*: combo, formal, other), and fuel type (*fuel_type*: gas, oil, electric, none, other). Binary features: garage exists (*garage_exists*), cats allowed (*cats_allowed*), dogs allowed (*dogs_allowed*). Missingness indicators: *miss_sq_footage, miss_monthly_cost, miss_parking_charges, miss_total_taxes, miss_num_floors_in_building, miss_pct_tax_deductibl, miss_dining_room_type, miss_fuel_type*.

Several features were engineered from raw columns. *ln_sq_footage* is a log transformation of square footage, motivated by the same distributional reasoning as the log response. *bldg_age* converts approximate year built into years of age relative to 2017. *bath_total* combines full and half bathrooms into a single numeric measure. *monthly_cost* merges two parallel columns - *maintenance_cost* for co-ops and *common_charges* for condos - into one, since each property type uses only one of the two.

Summary statistics for key continuous features in the 521-observation dataset: *ln_sale_price* mean 12.5, SD 0.556, range 10.9–13.8; *monthly_cost* mean $717, SD $366, range $148–$4,659; *num_bedrooms* mean 1.54, SD 0.75, range 0–3; *walk_score* mean 83.3, SD 12.9, range 15–99; *bldg_age* mean 54.8 years, SD 20.6. For categorical features: 75% of units are co-ops, 25% condos; *kitchen_type* is 44% efficiency, 40% eat in, 16% combo; *dining_room_type* is 63% combo, 26% formal, 10% other. 82% of observations have missing *pct_tax_deductibl* (a co-op-specific field), 74% have missing *total_taxes*, 75% have missing *parking_charges*, and 60% have missing square footage, making the missingness dummies for those features important.

2.2 Errors and Missingness

The raw data contained several obvious errors in the *kitchen_type* field, including "eatin", "efficiency ktchen", "efficiemcy", and "efficiency kitchene" - all standardized to their canonical values. Values not matching any valid category were set to NA. The *garage_exists* field contained non-standard entries including "ug", "underground", and "eys", all recoded as 1 (garage present).

Missingness is extensive and concentrated in a few key features. *ln_sq_footage* is missing for 60% of observations, *parking_charges* and *total_taxes* are each missing for approximately 75% of observations. *pct_tax_deductibl* is missing for 82%, though this is expected since it is co-op-specific. *num_floors_in_building* is missing for 21% of observations and *dining_room_type* for 22%.

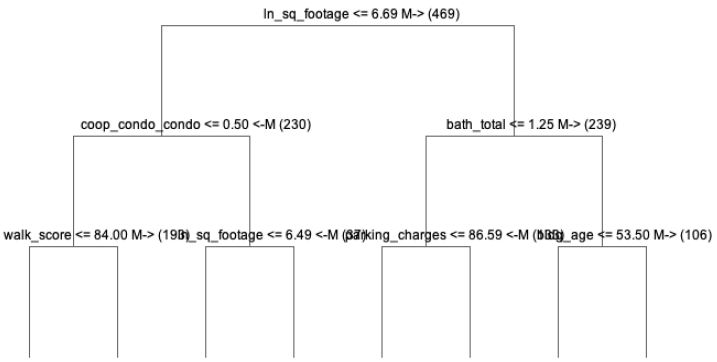
All missing values were imputed using missForest, which fits a separate random forest for each feature and iterates until convergence. Imputation was run on the full 2,230-row dataset prior to filtering to the 521-row sale population, following the principle that non-sale rows contain feature information that can improve imputation of the rows we actually model, even though they have no *y* value of their own. *sale_price* and *full_address_or_zip_code* were excluded from the imputation input.

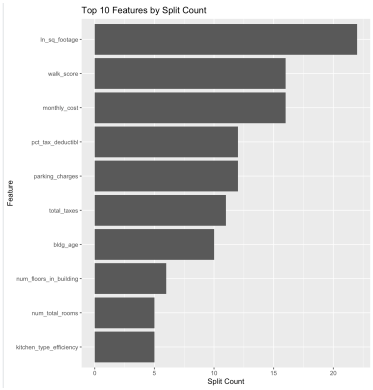
Missingness dummy variables were included for all columns with meaningful missingness rates. The rationale is that missingness in this dataset is not random - missing square footage likely indicates a data quality issue specific to certain listings, and the models should have the opportunity to learn whatever price signal is contained in that pattern. After full-dataset imputation, no further missingness remained in the modeling dataset.

3. Modeling

3.1 Regression Tree Modeling

A single regression tree was fit using YARFCART with default settings, trained on the 90% training split. The tree grew to 325 nodes with 163 leaves and a maximum depth of 21. The top 10 features by split count were: *ln_sq_footage* (24), *walk_score* (15), *monthly_cost* (14), *bldg_age* (11), *total_taxes* (11), *parking_charges* (11), *pct_tax_deductibl* (10), *num_floors_in_building* (7), *kitchen_type_efficiency* (5), and *num_total_rooms* (4). Notably, zip code dummies appear rarely in the tree despite being important in OLS - the tree finds that continuous features like square footage, walk score, and monthly cost capture enough of the relevant variation at the top levels. The neighborhood feature (north, jamaica, west, etc.) also appears, with *neighborhood_north* at 4 splits.





The tree achieves an in-sample R^2 of 0.961 and in-sample RMSE of 0.110, compared to an oos R^2 of 0.583 and oos RMSE of 0.354 (dollar-scale RMSE $102,900). The large gap between in-sample and oos performance indicates overfitting - the unpruned tree memorizes the training data almost perfectly but fails to generalize. This motivates the use of the random forest.

3.2 Linear Modeling

OLS was fit using $lm(y \sim ., train_df)$ with 86 columns after factor expansion and dummification. The model achieves an in-sample R^2 of 0.892 and in-sample *RMSE* of 0.183. The in-sample R^2 of 0.892 means the model explains 89% of the variance in log sale price on the training data. The in-sample *RMSE* of 0.183 log units corresponds to a dollar-scale *RMSE* of approximately $66,807; lower than the oos RMSE of $79,398, as expected since the model was fit on training data. The gap between in-sample and oos R^2 (0.892 vs 0.846) is modest, suggesting OLS is not severely overfitting despite the large number of features relative to observations.

The most important features from the regression tree are interpretable in the OLS output. The *zip_f11433* coefficient of −1.413 means units in that zip code sell for roughly 76% lower than the baseline price, holding all else equal - consistent with Jamaica being among the lowest-priced areas in Queens. The *coop_condocondo* coefficient of 0.795 means condos sell for $exp(0.795) \approx$ 2.2 times the price of otherwise similar co-ops, capturing the well-known condo premium over co-op ownership. The *ln_sq_footage* coefficient of 0.213 means a 1% increase in

square footage is associated with approximately a 0.213% increase in sale price - a reasonable size elasticity for apartment pricing. *num_bedrooms* has a coefficient of 0.167, meaning each additional bedroom is associated with roughly an 18% price increase holding other features constant. *monthly_cost* has a coefficient of 0.000281, meaning each additional dollar of monthly carrying cost is associated with a 0.028% price increase - a measure of how monthly costs are capitalized into sale price.

Variable	Estimate	Std. Error	t value	Pr(>|t|)	
(Intercept)	1.034e+01	4.937e-01	20.937	< 2e-16	***
coop_condocondo	7.945e-01	1.070e-01	7.424	7.01e-13	***
zip_f11433	-1.413e+00	2.188e-01	-6.458	3.11e-10	***
zip_f11414	-6.154e-01	8.347e-02	-7.373	9.85e-13	***
zip_f11368	-4.803e-01	1.011e-01	-4.752	2.82e-06	***
zip_f11101	3.146e-01	1.386e-01	2.271	0.0237	*
ln_sq_footage	2.132e-01	7.238e-02	2.946	0.0034	**
num_bedrooms	1.671e-01	2.613e-02	6.397	4.47e-10	***
monthly_cost	2.810e-04	5.037e-05	5.579	4.50e-08	***
num_total_rooms	4.416e-02	1.669e-02	2.646	0.0085	**
parking_charges	6.368e-04	3.230e-04	1.972	0.0493	*
dining_room_typeformal	5.991e-02	2.617e-02	2.289	0.0226	*

Most significant features

The neighborhood dummies were all dropped as NA due to perfect collinearity with the zip code factor, which fully spans the same geographic variation. With 469 observations and 86 features, OLS is already pushing its limits, but the signal happens to be linear enough that it works.

3.3 Random Forest Modeling

Random forest is a non-parametric method that fits many decorrelated regression trees on bootstrap samples of the training data and averages their predictions. It makes no assumptions about the functional form of the relationship between features and response. In theory, RF should outperform OLS here: it can capture interactions and non-linearities automatically, and averaging over 1,000 trees reduces variance substantially relative to a single tree.

With RF, you gain flexibility and robustness, and lose interpretability and efficiency at small sample sizes. The OOB error provides an honest in-sample estimate without needing a separate validation set, which is another practical advantage.

The RF does not appear to overfit - the OOB *RMSE* of 0.270 is very close to the oos *RMSE* of 0.273, meaning the forest generalizes about as well as it performs on held-out bootstrap samples. If anything, the model likely underfits slightly. The OOB *RMSE* bottoming out at mtry = 30 and ticking back up at mtry = 35 suggests moderate feature subsampling is optimal; the forest benefits from some decorrelation but not too much. This is more consistent with a partially non-linear signal than a purely linear one, though not enough non-linearity for RF to overcome the sample size disadvantage against OLS.

The top features by RF split count were *ln_sq_footage* (15,166), *monthly_cost* (14,576), *total_taxes* (13,970), *parking_charges* (13,531), *pct_tax_deductibl* (12,493), *walk_score*

(11,738), and *bldg_age* (11,250) - a ranking consistent with the single tree but with continuous features dominating even more strongly. The mtry parameter was tuned via OOB error across candidates {2, 3, 4, 7, 11, 16, 23, 30, 35}. OOB RMSE reached a minimum of 0.262 at mtry=30, with best_mtry selected as 30.

Despite tuning, the RF did not beat OLS. The tuned RF achieves oos R^2 of 0.754 and oos *RMSE* of 0.272 (dollar-scale $92,793), compared to OLS oos R^2 of 0.846 and oos *RMSE* of 0.215 ($79,398). The explanation is sample size. With 469 training observations and 86 features, each bootstrap sample contains roughly 300 unique observations. There is not enough data for RF to reliably learn non-linear structure across this feature space. The additional flexibility from capturing interactions and non-linearities does not seem to outweigh the variance cost at this sample size. OLS, which estimates all coefficients in one pass, is more efficient in this low-n, high-p setting. The modeling process was iterative - features were added and removed, missingness dummies were tuned, and different imputation strategies were explored - but the relative ordering of models was consistent throughout.

It is worth noting that some classmates reported RF outperforming OLS on the same dataset, which suggests feature representation may play a role. The 42-level zip code factor expands to 41 binary dummies that RF must learn location effects from through repeated splits - inefficient compared to OLS which gets one coefficient per zip directly. Binning zip codes into 9 neighborhood regions was attempted but hurt overall performance, suggesting the precise location signal is worth preserving. Exploring neighborhood binning in combination with different featurization might find a configuration where RF benefits from the broader geography without losing the location signal, which may be worth exploring in the future.

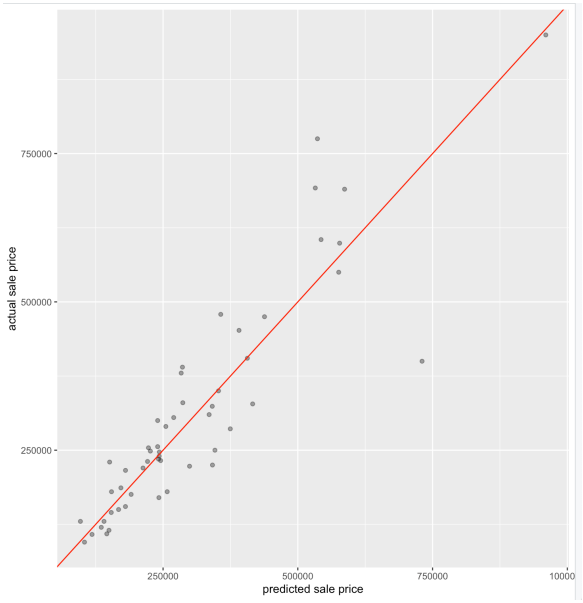
4. Performance Results

Model	In-Sample R^2	In-Sample RMSE	OOB RMSE	OOS R^2	OOS RMSE (log)	OOS RMSE ($)
OLS	0.892	0.183	—	0.846	0.215	$79,397.73
Regression Tree	0.961	0.110	—	0.583	0.354	$102,900.47
RF (default)	—	—	0.270	0.752	0.273	$92,613.64
RF (tuned, mtry=30)	—	—	0.262	0.754	0.272	$92,792.87

Within OLS, the in-sample to oos gap is small (R^2 0.892 vs 0.846, *RMSE* 0.183 vs 0.215), suggesting the model generalizes well relative to its complexity. The tree shows the opposite pattern; near-perfect in-sample fit (R^2 0.961) falling to 0.583 oos, a large overfit. Between models, OLS does best on every oos metric despite being the simplest of the three. The tuned RF improves on the default in log-scale *RMSE* (0.272 vs 0.273) and OOB *RMSE* (0.262 vs 0.270), while doing slightly worse on the dollar-scale RMSE, though the difference is negligible - neither version comes close to OLS. The dollar-scale *RMSE* is the most interpretable metric here; OLS is off by roughly $79,400 on average, the tuned RF by $92,800, and the tree by $102,900. Comparing to the Zillow benchmark:

Metric	Our Model (*OLS*)	Zillow
Median Error	12.1%	2.3%
Within 5%	19.2%	80.2%
Within 10%	42.3%	94.1%
Within 20%	65.4%	98.4%

The gap is substantial but expected. The oos estimates are valid proxies for future performance because the test set was held out entirely during training - a random 10% sample drawn with set.seed(1984) from the same population as the training data. The consistency of the OOB estimate with oos RMSE (0.270 vs 0.273) further validates that the test set estimate is not just a lucky draw.



This figure shows predicted versus actual sale prices for the best model (OLS). Points near the red diagonal indicate accurate predictions. The model tracks well at lower price points below $300K where most co-ops cluster, but shows more scatter and slight underprediction in the $400K–$800K range. The one point near $1M actual is predicted accurately. The spread at higher prices possibly reflects features of high-end units that aren't in the dataset, as well as the limited number of more expensive units in the dataset.

5. Discussion

The central finding is that OLS outperforms random forest on this dataset, which runs counter to typical expectations. The explanation - small *n*, predominantly linear signal, high missingness in key continuous features – shows that simpler models can remain competitive.

The most significant limitation is sample size. 521 total observations, 469 in training, is not enough to train a random forest that reliably learns non-linear structure across 86 features. This is a dataset-level constraint that cannot be addressed through modeling choices alone. The

OOB RMSE reached a minimum at mtry = 30, suggesting moderate feature subsampling is optimal: the forest benefits from some decorrelation but not too much, which is more consistent with a partially non-linear signal than a purely linear one.

The 60% missingness in square footage is the second major limitation. Square footage is the most important continuous feature in the RF (15,166 splits) and the second most important in OLS, yet the majority of values are imputed rather than observed. Better data collection from public records would dramatically improve model quality.

Future extensions could include external data sources such as public records for square footage and tax assessments, proximity measures to subway stations and commercial areas, and a larger dataset covering multiple years to allow seasonal controls. A larger sample would likely close the gap between OLS and RF and improve absolute performance across all models.

The model is not production ready in the sense of competing with Zillow. A 12.1% median error versus Zillow's 2.3% is a large gap, and only 19.2% of predictions fall within 5% compared to Zillow's 80.2%. At the current dataset size and quality level, the model is not competitive with Zillow - but given 521 noisy observations with 60% missingness in the most important feature, the model doesn’t perform badly.

```r
pacman::p_load(data.table, ggplot2, skimr, missForest, forcats)


options(java.parameters = "-Xmx8000m")
pacman::p_load(rJava)
.jinit()


if (!pacman::p_isinstalled(YARF)){
  pacman::p_install_gh("kapelner/YARF/YARFJARs", ref = "dev")
  pacman::p_install_gh("kapelner/YARF/YARF", ref = "dev", force = TRUE)
}
pacman::p_load(YARF)


housing = fread("housing_data.csv")
dim(housing)
head(housing)
skim(housing)


housing = housing[, .(
  coop_condo, full_address_or_zip_code, approx_year_built, cats_allowed, dogs_allowed, common_charges,
maintenance_cost,
  parking_charges, dining_room_type, fuel_type, garage_exists, kitchen_type, num_bedrooms,
num_floors_in_building,
  num_full_bathrooms, num_half_bathrooms, num_total_rooms, pct_tax_deductibl, sale_price, sq_footage,
total_taxes, walk_score
)]


housing[, approx_year_built := as.integer(approx_year_built)]
housing[, coop_condo := factor(coop_condo)]
housing[, sq_footage := as.numeric(gsub("[^0-9.]", "", sq_footage))]


housing[, parking_charges := as.numeric(gsub("[$, ]", "", parking_charges))]


housing[, total_taxes := as.numeric(gsub("[$, ]", "", total_taxes))]
housing[, pct_tax_deductibl := as.numeric(pct_tax_deductibl)]


housing[, maintenance_cost := as.numeric(gsub("[$, ]", "", maintenance_cost))]
housing[, common_charges := as.numeric(gsub("[$, ]", "", common_charges))]
housing[, monthly_cost := ifelse(!is.na(maintenance_cost), maintenance_cost, common_charges)]
housing[, num_bedrooms := as.integer(num_bedrooms)]
housing[, num_total_rooms := as.integer(num_total_rooms)]
housing[, num_floors_in_building := as.integer(num_floors_in_building)]
housing[, num_full_bathrooms := as.integer(num_full_bathrooms)]
housing[, num_half_bathrooms := as.integer(num_half_bathrooms)]


housing[, bath_total := num_full_bathrooms + 0.5 * ifelse(is.na(num_half_bathrooms), 0, num_half_bathrooms)]


housing[, kitchen_type := tolower(trimws(kitchen_type))]
housing[kitchen_type %in% c("eatin", "eat in"), kitchen_type := "eat in"]
housing[kitchen_type %in% c("efficiency kitchen", "efficiency ktchen", "efficiemcy", "efficiency kitchene"),
kitchen_type := "efficiency"]
housing[!(kitchen_type %in% c("eat in", "efficiency", "combo", "none")), kitchen_type := NA]
housing[, kitchen_type := factor(kitchen_type)]


housing[, dining_room_type := tolower(trimws(dining_room_type))]
housing[dining_room_type == "na", dining_room_type := NA]
housing[dining_room_type %in% c("dining area"), dining_room_type := "other"]
housing[, dining_room_type := factor(dining_room_type)]


housing[, fuel_type := tolower(trimws(fuel_type))]
housing[fuel_type == "na", fuel_type := NA]
housing[, fuel_type := factor(fuel_type)]


housing[, garage_exists := tolower(trimws(garage_exists))]
```

```
housing[, garage_exists := ifelse(garage_exists %in% c("na", ""), NA, ifelse(garage_exists %in% c("yes", "1",
"ug", "underground", "eys"), 1, 0))]
housing[, garage_exists := as.integer(garage_exists)]


housing[, cats_allowed := as.integer(tolower(trimws(cats_allowed)) %in% c("yes", "y"))]
housing[, dogs_allowed := as.integer(tolower(trimws(dogs_allowed)) %in% c("yes", "y", "yes89"))]


housing[, miss_sq_footage := as.integer(is.na(sq_footage))]
housing[, miss_monthly_cost := as.integer(is.na(monthly_cost))]
housing[, miss_parking_charges := as.integer(is.na(parking_charges))]
housing[, miss_total_taxes := as.integer(is.na(total_taxes))]
housing[, miss_num_floors_in_building := as.integer(is.na(num_floors_in_building))]
housing[, miss_pct_tax_deductibl := as.integer(is.na(pct_tax_deductibl))]
housing[, miss_dining_room_type := as.integer(is.na(dining_room_type))]
housing[, miss_fuel_type := as.integer(is.na(fuel_type))]


sale_price_raw = housing$sale_price
address_raw = housing$full_address_or_zip_code
set.seed(1984)
housing_imp = missForest(data.frame(housing[, !c("sale_price", "full_address_or_zip_code")]))$ximp
housing = as.data.table(housing_imp)
housing[, sale_price := sale_price_raw]
housing[, full_address_or_zip_code := address_raw]



housing[, sale_price := as.numeric(gsub("[$, ]", "", sale_price))]
housing = housing[!is.na(sale_price)]


housing[, zip := as.integer(regmatches(
  full_address_or_zip_code,
  regexpr("[0-9]{5}", full_address_or_zip_code)
))]


valid_zips = c(
  11361, 11362, 11363, 11364, 11354, 11355, 11356, 11357, 11358, 11359, 11360, 11365, 11366, 11367, 11412, 11423,
  11432, 11433, 11434, 11435, 11436, 11101, 11102, 11103, 11104, 11105, 11106, 11374, 11375, 11379, 11385, 11004,
  11005, 11411, 11413, 11422, 11426, 11427, 11428, 11429, 11414, 11415, 11416, 11417, 11418, 11419, 11420, 11421,
  11368, 11369, 11370, 11372, 11373, 11377, 11378
)

housing = housing[zip %in% valid_zips]
housing = housing[sale_price <= 1e6]
dim(housing)


ggplot(housing) +
  aes(x = sale_price) +
  geom_histogram(bins = 50)


ggplot(housing) +
  aes(x = log(sale_price)) +
  geom_histogram(bins = 50)


housing[, ln_sale_price := log(sale_price)]


housing[, zip_f := factor(zip)]
housing[, neighborhood := ifelse(zip %in% c(11361, 11362, 11363, 11364), "northeast",
                          ifelse(zip %in% c(11354, 11355, 11356, 11357, 11358, 11359, 11360), "north",
                          ifelse(zip %in% c(11365, 11366, 11367), "central",
                          ifelse(zip %in% c(11412, 11423, 11432, 11433, 11434, 11435, 11436), "jamaica",
                          ifelse(zip %in% c(11101, 11102, 11103, 11104, 11105, 11106), "northwest",
                          ifelse(zip %in% c(11374, 11375, 11379, 11385), "west_central",
                          ifelse(zip %in% c(11004, 11005, 11411, 11413, 11422, 11426, 11427, 11428, 11429),
"southeast",
                          ifelse(zip %in% c(11414, 11415, 11416, 11417, 11418, 11419, 11420, 11421), "southwest",
                                                                                    "west")))))))]
housing[, neighborhood := factor(neighborhood)]
```

```r
housing[, ln_sq_footage := log(sq_footage)]
housing[, bldg_age := 2017 - approx_year_built]


modeling_cols = c(
  "ln_sale_price", "coop_condo", "zip_f", "neighborhood", "ln_sq_footage", "bldg_age", "num_bedrooms",
"bath_total", "num_total_rooms",
  "num_floors_in_building", "monthly_cost", "total_taxes", "walk_score", "parking_charges",
"miss_parking_charges", "miss_total_taxes", "pct_tax_deductibl",
  "kitchen_type", "dining_room_type", "fuel_type", "garage_exists", "cats_allowed", "dogs_allowed",
"miss_sq_footage",
  "miss_monthly_cost", "miss_num_floors_in_building", "miss_pct_tax_deductibl", "miss_dining_room_type",
"miss_fuel_type"
)


cleaned_dataset = housing[, ..modeling_cols]
dim(cleaned_dataset)
skim(cleaned_dataset)


y_all = cleaned_dataset$ln_sale_price
X_all = cleaned_dataset[, !"ln_sale_price"]


X_imp = as.data.table(data.frame(X_all))


set.seed(1984)
n = nrow(X_imp)
test_indices = sample(1 : n, round(0.1 * n))
train_indices = setdiff(1 : n, test_indices)


X_train = X_imp[train_indices, ]
y_train = y_all[train_indices]
X_test = X_imp[test_indices, ]
y_test = y_all[test_indices]



# Regression Tree
tree_mod = YARFCART(X_train, y_train, calculate_oob_error = FALSE, seed = 1984)


get_tree_num_nodes_leaves_max_depths(tree_mod)
illustrate_trees(tree_mod, max_depth = 4, open_file = TRUE)


var_counts_tree = query_variable_counts(tree_mod)
top10 = sort(var_counts_tree, decreasing = TRUE)[1:10]
ggplot(data.frame(variable = fct_reorder(names(top10), top10), count = top10)) +
  geom_col(aes(x = variable, y = count)) +
  coord_flip() +
  xlab("Feature") + ylab("Split Count") +
  ggtitle("Top 10 Features by Split Count")
sort(query_variable_counts(tree_mod), decreasing = TRUE)


yhat_train_tree = predict(tree_mod, X_train)
e_train_tree = y_train - yhat_train_tree
tree_in_Rmse = sd(e_train_tree)
tree_in_Rsq = 1 - sum(e_train_tree^2) / sum((y_train - mean(y_train))^2)
tree_in_Rmse_dollars = sqrt(mean((exp(y_train) - exp(yhat_train_tree))^2))
cat("tree in-sample RMSE:", tree_in_Rmse, "\n")
cat("tree in-sample Rsq:", tree_in_Rsq, "\n")
cat("tree in-sample RMSE ($):", tree_in_Rmse_dollars, "\n")


yhat_test_tree = predict(tree_mod, X_test)
e_test_tree = y_test - yhat_test_tree
tree_oos_Rmse = sd(e_test_tree)
tree_oos_Rsq = 1 - sum(e_test_tree^2) / sum((y_test  - mean(y_test))^2)
tree_oos_Rmse_dollars = sqrt(mean((exp(y_test) - exp(yhat_test_tree))^2))
cat("tree oos RMSE:", tree_oos_Rmse, "\n")
cat("tree oos Rsq:", tree_oos_Rsq, "\n")
cat("tree oos RMSE ($):", tree_oos_Rmse_dollars, "\n")
```

```r
# OLS
train_df = data.frame(y = y_train, X_train)
ols_mod = lm(y ~ ., train_df)
summary(ols_mod)


e_train_ols = y_train - ols_mod$fitted.values
ols_in_Rmse = sd(e_train_ols)
ols_in_Rsq = summary(ols_mod)$r.squared
ols_in_Rmse_dollars = sqrt(mean((exp(y_train) - exp(ols_mod$fitted.values))^2))
cat("OLS in-sample RMSE:", ols_in_Rmse, "\n")
cat("OLS in-sample Rsq:", ols_in_Rsq, "\n")
cat("OLS in-sample RMSE ($):", ols_in_Rmse_dollars, "\n")


yhat_test_ols = predict(ols_mod, data.frame(X_test))
e_test_ols = y_test - yhat_test_ols
ols_oos_Rmse = sd(e_test_ols)
ols_oos_Rsq = 1 - sum(e_test_ols^2) / sum((y_test - mean(y_test))^2)
ols_oos_Rmse_dollars = sqrt(mean((exp(y_test) - exp(yhat_test_ols))^2))
cat("OLS oos RMSE:", ols_oos_Rmse, "\n")
cat("OLS oos Rsq:", ols_oos_Rsq, "\n")
cat("OLS oos RMSE ($):", ols_oos_Rmse_dollars, "\n")


# Random Forest
rf_mod = YARF(X_train, y_train, num_trees = 1000, seed = 1984)
rf_mod


yhat_test_rf = predict(rf_mod, X_test)
e_test_rf = y_test - yhat_test_rf
rf_oos_Rmse = sd(e_test_rf)
rf_oos_Rsq = 1 - sum(e_test_rf^2) / sum((y_test - mean(y_test))^2)
rf_oos_Rmse_dollars = sqrt(mean((exp(y_test) - exp(yhat_test_rf))^2))
cat("RF oos RMSE:", rf_oos_Rmse, "\n")
cat("RF oos Rsq:", rf_oos_Rsq, "\n")
cat("RF oos RMSE ($):", rf_oos_Rmse_dollars, "\n")


var_counts_rf = query_variable_counts(rf_mod)
ggplot(data.frame(variable = fct_reorder(names(var_counts_rf), var_counts_rf), count = var_counts_rf)) +
  geom_col(aes(x = variable, y = count)) +
  coord_flip()
sort(query_variable_counts(rf_mod), decreasing = TRUE)


# mtry tuning
p = ncol(X_train)
mtry_candidates = c(2, 3, 4, 7, 11, 16, 23, 30, 35)
oob_by_mtry = numeric(length(mtry_candidates))
for (i in seq_along(mtry_candidates)){
  mod_tmp = YARF(X_train, y_train, num_trees = 1000, mtry = mtry_candidates[i], verbose = FALSE, seed = 1984)
  oob_by_mtry[i] = mod_tmp$rmse_oob
  cat("mtry =", mtry_candidates[i], " oob rmse =", round(oob_by_mtry[i], 4), "\n")
}
best_mtry = mtry_candidates[which.min(oob_by_mtry)]


rf_tuned = YARF(X_train, y_train, num_trees = 1000, mtry = best_mtry, seed = 1984)
yhat_test_rf_tuned = predict(rf_tuned, X_test)
e_test_rf_tuned = y_test - yhat_test_rf_tuned
rft_oos_Rmse = sd(e_test_rf_tuned)
rft_oos_Rsq = 1 - sum(e_test_rf_tuned^2) / sum((y_test - mean(y_test))^2)
rft_oos_Rmse_dollars = sqrt(mean((exp(y_test) - exp(yhat_test_rf_tuned))^2))
cat("RF tuned oos RMSE:", rft_oos_Rmse, "\n")
cat("RF tuned oos Rsq:", rft_oos_Rsq, "\n")
cat("RF tuned oos RMSE ($):", rft_oos_Rmse_dollars, "\n")



ggplot(data.frame(actual = exp(y_test), predicted = exp(yhat_test_rf_tuned))) +
  geom_point(aes(x = predicted, y = actual), alpha = 0.4) +
  geom_abline(slope = 1, intercept = 0, col = "red") +
  xlab("predicted sale price") + ylab("actual sale price")
```

```r
results = data.frame(
  model = c("OLS", "Tree", "RF", "RF tuned"),
  in_sample_rsq = c(ols_in_Rsq, tree_in_Rsq, NA, NA),
  in_sample_rmse = c(ols_in_Rmse, tree_in_Rmse, NA, NA),
  oob_rmse = c(NA, NA, rf_mod$rmse_oob, rf_tuned$rmse_oob),
  oos_rsq = c(ols_oos_Rsq, tree_oos_Rsq, rf_oos_Rsq, rft_oos_Rsq),
  oos_rmse = c(ols_oos_Rmse, tree_oos_Rmse, rf_oos_Rmse, rft_oos_Rmse),
  oos_rmse_dollars = c(ols_oos_Rmse_dollars, tree_oos_Rmse_dollars, rf_oos_Rmse_dollars, rft_oos_Rmse_dollars)
)
results


best_model = which.min(c(ols_oos_Rmse, tree_oos_Rmse, rf_oos_Rmse, rft_oos_Rmse))
cat("Best model:", c("OLS", "Tree", "RF", "RF tuned")[best_model], "\n")


best_predictor = list(yhat_test_ols, yhat_test_tree, yhat_test_rf, yhat_test_rf_tuned)[[best_model]]
oos_preds = exp(best_predictor)
y_true = exp(y_test)


abs_errors = abs(y_true - oos_preds)
median_pct_error = median(abs_errors / y_true) * 100


pct_within_5 = mean(abs_errors / y_true < 0.05) * 100
pct_within_10 = mean(abs_errors / y_true < 0.10) * 100
pct_within_20 = mean(abs_errors / y_true < 0.20) * 100


cat("Median Error:", round(median_pct_error, 1), "%\n")
cat("Within 5%:", round(pct_within_5, 1), "%\n")
cat("Within 10%:", round(pct_within_10, 1), "%\n")
cat("Within 20%:", round(pct_within_20, 1), "%\n")


final_mod = list(ols_mod, tree_mod, rf_mod, rf_tuned)[[best_model]]
final_mod
```

ChatGPT colored